

Ejercicio 3: Partición Óptima de Líneas (Análisis del Algoritmo Voraz)

Enunciado

Se nos proporciona una lista ordenada de n palabras. La longitud de la palabra que ocupa la posición i en la lista es w_i . Necesitamos romper la lista de palabras en líneas, sin cambiar su orden relativo.

La longitud de una línea es la suma de las longitudes de las palabras de esa línea. La longitud ideal de una línea es L . Ninguna línea puede ocupar más de L , pero sí que pueden ser más cortas. La penalización por tener una línea de longitud K es $L - K$. La penalización total para una distribución de palabras es la suma de las penalizaciones de cada línea.

Debemos diseñar una distribución de palabras que minimice la penalización total.

Mostrar o encontrar un contraejemplo sobre si el siguiente algoritmo voraz resuelve el problema: *“Colocar cada una de las palabras en la línea actual si cabe; si no, colocarla en la línea siguiente”*.

1. Análisis Algebraico de la Penalización (La clave del problema)

Para evaluar si el algoritmo voraz funciona, primero simplificamos matemáticamente la penalización total (P_{total}).

Supongamos que una distribución válida cualquiera divide las n palabras en un total de m líneas. Sean $K_1, K_2, \dots, K_m$ las longitudes de cada una de estas m líneas.

La penalización se calcula sumando las diferencias individuales de cada línea:

$$P_{\text{total}} = \sum_{i=1}^m (L - K_i)$$

Separando los términos del sumatorio:

$$P_{\text{total}} = \sum_{i=1}^m L - \sum_{i=1}^m K_i$$

Como L es una constante (la longitud máxima de cada línea):

$$\sum_{i=1}^m L = m \cdot L$$

Por otro lado, la suma de las longitudes de todas las líneas ($\sum K_i$) tiene que ser exactamente igual a la suma de las longitudes de todas las palabras de la lista (ya que no hay espacios intermedios y todas las palabras se deben colocar). Definimos esta suma constante de palabras como S :

$$\sum_{i=1}^m K_i = \sum_{j=1}^n w_j = S \quad (\text{constante})$$

Sustituyendo esto en la fórmula original, la penalización total queda como:

$$P_{\text{total}} = m \cdot L - S$$

Conclusión:

Dado que L y S son valores fijos y constantes para una entrada dada, la penalización **depende únicamente del número de líneas m que usemos**. Por tanto:

$$\text{Minimizar la penalización } P_{\text{total}} \iff \text{Minimizar el número de líneas } m$$

2. Justificación Intuitiva de la Correctitud

Para demostrar que el algoritmo voraz es óptimo, solo tenemos que probar que **minimiza el número total de líneas m utilizadas**.

Explicación del comportamiento voraz:

El algoritmo voraz empaqueta tantas palabras como físicamente quepan en la línea actual antes de abrir la siguiente. Al hacer esto, el voraz va "empujando" las palabras lo más a la derecha posible en cada línea.

Si comparamos la distribución del algoritmo voraz con **cualquier otra distribución válida**:

1. **En la línea 1:** El voraz mete el máximo número posible de palabras. Cualquier otra distribución meterá, como mucho, esa misma cantidad de palabras o menos en su primera línea.
2. **En la línea 2:** El voraz comenzará a colocar palabras desde una posición igual o más avanzada en la lista que la otra distribución. Como tiene menos palabras restantes por colocar y el mismo límite de espacio L , al final de la segunda línea el voraz seguirá habiendo colocado acumulativamente tantas o más palabras que la otra distribución.
3. **En general:** Al final de cualquier línea i , el algoritmo voraz siempre habrá logrado colocar en total **tantas o más palabras** de la lista que cualquier otra distribución alternativa.

Conclusión:

Dado que el voraz siempre va por delante (o empatado) en número de palabras colocadas al final de cada línea, terminará de colocar las n palabras en **el menor número posible de líneas** m .

Como minimizar m equivale directamente a minimizar la penalización total (demostrado en la Sección 1), **el algoritmo voraz es óptimo y resuelve el problema.**

3. Justificación del Coste en Tiempo

El coste de este algoritmo voraz es extremadamente eficiente:

- **Temporal:** $\mathcal{O}(n)$. Solo hacemos una única pasada lineal sobre la lista de las n palabras. Para cada palabra realizamos una comprobación de coste constante $\mathcal{O}(1)$ (ver si cabe en el espacio restante de la línea actual).
- **Espacial:** $\mathcal{O}(1)$ de memoria auxiliar, ya que solo necesitamos llevar un contador de espacio restante en la línea actual e ir imprimiendo las palabras o los saltos de línea según